RAPPORT FINAL DE QUALIFICATION — CreditRisk-Test

https://github.com/kagibru/CreditRisk-Test/

1. Synthèse exécutive

La campagne de qualification porte sur l'API de prédiction du risque de défaut de crédit : validation des données, fonctionnalités HTTP, intégration du pipeline, robustesse, non-régression, performance et intégration continue. La dernière exécution locale sous Windows et Python 3.13.2 a confirmé 80 tests réussis sur 80 en 3,67 secondes, sans échec, avec un avertissement non bloquant. Ruff a également été exécuté sans erreur.

La couverture globale mesurée est de 91 %. Les quatre campagnes Locust totalisent 10 418 requêtes sans échec, avec un P95 de 20 à 21 ms. Une démonstration antérieure a prouvé qu'un test Pytest volontairement défaillant entraîne l'échec du workflow GitHub Actions (56 tests réussis, 1 échec intentionnel).

Avis : favorable à une démonstration pédagogique ; défavorable à une mise en production ou à une utilisation pour des décisions financières réelles. Le rappel (recall) de 14,65 % sur données synthétiques est insuffisant pour établir l'aptitude métier.

2. Système évalué et périmètre

Le système repose sur FastAPI et scikit-learn, avec un modèle de régression logistique entraîné sur des données synthétiques. Les points d'accès évalués sont /health, /model-info et /predict. Le périmètre inclut validation, prétraitement, chargement du modèle, inférence, réponses HTTP, gestion des erreurs et reproductibilité.

Hors périmètre : validation sur dossiers réels, calibration, équité entre groupes, surveillance de dérive, sécurité opérationnelle, endurance et qualification d'une infrastructure de production.

3. Environnement, méthode et preuves

Domaine	Environnement / méthode	Preuves
Tests locaux	Windows 11 Pro 64 bits ; Python 3.13.2 ; Pytest	80/80 réussis ; exécution locale du 10/11/2026
Machine de test	Intel Core Ultra 7 165H ; 16 cœurs, 22 processeurs logiques ; 63,45 Go RAM	Configuration locale documentée
API et modèle	FastAPI 0.162.2 ; scikit-learn 1.9.1	tests/ ; scripts/train_model.py
Charge	Locust 2.47.0 ; campagnes HTTP locales	reports/load_*_stats.csv
CI	GitHub Actions ; Ubuntu ; Python 3.12 ; Ruff ; pytest-cov	Run de démonstration #42700994044
Traçabilité	Plan de test, matrice, registre d'anomalies	reports/plan_de_test.md ; reports/matrice_tracabilite.md

Les versions Python diffèrent entre le poste local (3.13.2) et la CI (3.12). Les mesures TestClient et Locust relèvent de protocoles distincts.

4. Résultats des tests automatisés

Catégorie	Réussis	Échecs
Fonctionnels (dont 24 nouveaux scénarios aux limites)	41	0
Intégration	11	0
Robustesse	18	0
Non-régression (test_regression.py)	7	0
Performance automatisée	2	0
Métriques du modèle	1	0

TOTAL	80	0

Les contrôles de non-régression représentent huit tests si l'on inclut le test distinct des métriques du modèle ; ce dernier n'est pas additionné deux fois. Les 24 scénarios ajoutés couvrent les bornes d'âge, de revenu, de montant de crédit, de durée et d'ancienneté. La couverture globale reste de 91 % ; un avertissement StarletteDeprecationWarning demeure non bloquant.

5. Résultats de performance

5.1 Mesures locales TestClient

Indicateur	Mesure
Latence API moyenne	4,75 ms
Latence API P95	5,53 ms
Débit séquentiel estimé	210,60 req/s
Latence d'inférence P95	0,36 ms

5.2 Campagnes de charge Locust

Utilisateurs	Requêtes	Échecs	Débit (req/s)	P95	P99
5	191	0	3,24	20 ms	810 ms
20	767	0	12,90	20 ms	24 ms
50	1 895	0	31,88	21 ms	31 ms
100	7 565	0	63,62	20 ms	30 ms
TOTAL	10 418	0	—	—	—

Les scénarios à 5, 20 et 50 utilisateurs étaient programmés pour 60 secondes ; celui à 100 utilisateurs pour 120 secondes, avec un temps d'attente aléatoire de 1 à 2 secondes. Les débits ne s'additionnent pas. Un pic atypique de 1 818,21 ms à cinq utilisateurs demeure inexpliqué.

Critères pédagogiques proposés : 0 % d'échec, P95 inférieur à 100 ms et débit d'au moins 50 req/s à 100 utilisateurs. Ces critères ont été satisfaits dans les conditions locales testées, sans constituer des objectifs contractuels de production.

6. Traçabilité et non-régression

La matrice de traçabilité documente 20 exigences techniques et leur correspondance avec les sept exigences EX01–EX07 du référentiel. Les tests fonctionnels couvrent désormais les valeurs immédiatement hors limites et les bornes autorisées pour les principales variables.

Le contrôle test_model_metrics_against_baseline compare les métriques du modèle courant à la baseline : recall courant et de référence = 0,1465 ; F1 courant et de référence = 0,2473. La tolérance absolue est de 0,02 par métrique, sur un jeu synthétique reproductible de 1 000 observations (seed 42). Deux tests vérifient la détection de baisses simulées de cinq points de pourcentage.

Limite : aucune comparaison de deux artefacts de modèles distincts (V1/V2) n'a été démontrée. L'exigence EX-19 demeure partiellement vérifiée.

7. Anomalies, risques et réserves

Observation	Constat	Suite recommandée
Robustesse	18 tests réussis ; 24 scénarios de limites supplémentaires validés	Conserver le registre et documenter les réponses observées
Dépendances	Avertissement StarletteDeprecationWarning	Surveiller les mises à jour
Latence atypique	Maximum 1 818,21 ms à 5 utilisateurs	Répéter et instrumenter la mesure
Non-régression	Baseline vérifiée ; V1/V2 non	Comparer deux artefacts versionnés si

	comparées	requis
Qualité prédictive	Recall 14,65 % ; F1 24,73 % ; données synthétiques	Évaluer faux négatifs, calibration et biais
Couverture et charge	91 % ; absence de suivi CPU/RAM et d'endurance	Compléter l'observabilité

Un rappel de 14,65 % signifie qu'environ 85,35 % des défauts présents dans ce jeu de validation ne sont pas détectés, si la classe positive désigne le défaut. La qualité des tests logiciels ne garantit pas la pertinence des décisions de crédit.

8. Intégration continue et décision de qualification

Le workflow GitHub Actions est déclenché par push, pull_request ou manuellement. Il installe Python 3.12 et les dépendances, entraîne le modèle synthétique, exécute Ruff et Pytest avec couverture, puis archive les rapports XML. La démonstration de blocage CI sur la branche temporaire demo/ci-test-failure a produit 56 passed, 1 failed, 1 warning et un code de sortie 1 : https://github.com/kagibru/CreditRisk-Test/actions/runs/42700994044. Cette branche n'a pas été fusionnée dans main.

Critère	Résultat	Appréciation
Tests locaux sur main	80/80 ; Ruff conforme	Satisfait localement
Couverture locale	91 %	Satisfait (seuil pédagogique 80 %)
CI après ajout des 24 cas	Nouvelle exécution non vérifiée dans ce rapport	À confirmer
Échec CI sur test défaillant	Run 42700994044 ; exit code 1	Démontré
Charge locale	10 418 requêtes ; 0 échec ; P95 20–21 ms	Satisfait localement
Non-régression	Baseline contrôlée ; V1/V2 non comparées	Partiellement démontré
Production bancaire	Données synthétiques ; recall faible	Non qualifié

DÉCISION : qualification favorable pour une démonstration pédagogique du système de tests et de l'API ; qualification défavorable pour toute utilisation opérationnelle dans une institution financière. La réussite technique ne vaut pas validation de la pertinence des décisions de crédit.

9. Recommandations et clôture

- Conserver les preuves reproductibles : journaux GitHub Actions, rapports XML, couverture, CSV Locust, matrice et registre des anomalies.
- Comparer deux artefacts de modèles distincts sur le même jeu de validation figé, avec seuils documentés, lors d'une évolution du modèle.
- Répéter les campagnes de charge, investiguer le pic de latence et instrumenter CPU, RAM et temps de réponse.
- Avant tout usage métier : valider sur données réelles représentatives, analyser faux négatifs, calibration, biais, dérive et supervision humaine.
- Mettre à jour les dépendances de manière contrôlée et conserver la preuve que la CI bloque un test défaillant.

Références internes

- reports/plan_de_test.md
- reports/matrice_tracabilite.md
- reports/registre_anomalies_robustesse.md
- reports/rapport_non_regression_creditrisk.md
- reports/rapport_performance_creditrisk.md
- reports/load_5_stats.csv, load_20_stats.csv, load_50_stats.csv, load_100_stats.csv
- reports/coverage.xml et rapports XML Pytest
- GitHub Actions : run 42700994044